

# Clase para estudiar para el quiz de algoritmo y razonamiento computacional

## TypeScript: JavaScript con Superpoderes

### ¿Qué es TypeScript?

**TypeScript (TS)** es un lenguaje de programación de código abierto desarrollado por **Microsoft**. Se define como un **superset** (superconjunto) de JavaScript. Esto significa que:

1. Todo el código de JavaScript es código de TypeScript válido.
2. TS añade una capa de **tipado estático** y características avanzadas que no existen en JS puro.

#### Important

**TS no se ejecuta en el navegador.** Los navegadores (como Firefox, Chrome o Safari) solo entienden JavaScript. Por eso, el código TS debe ser **compilado** (o transpilado) a JS antes de desplegarse.

---

## Un poco de Historia

TypeScript nació en las oficinas de **Microsoft** y fue lanzado oficialmente en **2012**. Su creador es **Anders Hejlsberg**, el mismo arquitecto detrás de lenguajes como C# y Delphi.

### ¿Por qué se creó?

A medida que las aplicaciones web crecían, el código de JavaScript se volvía difícil de mantener. En proyectos gigantes, era casi imposible saber qué tipo de datos recibía una función, lo que causaba errores constantes en tiempo de ejecución. TS se diseñó para escalar el desarrollo de JS a nivel empresarial.

---

## Popularidad y Datos (2026)

La adopción de TypeScript ha sido explosiva. Según datos de la industria:

- **Adopción en Empresas:** Más del **85%** de los nuevos proyectos a gran escala en empresas tecnológicas (Fintech, SaaS, E-commerce) eligen TS sobre JS.
- **Comunidad:** Se mantiene en el **Top 3** de lenguajes más amados y usados según encuestas de desarrolladores.

- **Mercado Laboral:** Dominar TS ya no es un "extra", es un requisito estándar para roles de Senior Developer.

---

## Diferencias Clave: JS vs. TS

### 1. El Sistema de Tipos

| Característica | JavaScript (JS)                   | TypeScript (TS)                        |
|----------------|-----------------------------------|----------------------------------------|
| Tipado         | Dinámico y Débil                  | Estático y Fuerte                      |
| Errores        | Se detectan al ejecutar (Runtime) | Se detectan al escribir (Compile time) |
| Escalabilidad  | Difícil en proyectos grandes      | Diseñado para grandes bases de código  |
| Documentación  | Implícita (hay que adivinar)      | Autodocumentado por los tipos          |

### 2. Ejemplo Práctico de Tipado

En **JavaScript**, esto es válido pero peligroso:

JavaScript

```
let a = 'hola'; // Es un string
a = 2;           // ¡Ahora es un número! JS no se queja.
console.log(typeof a); // Output: "number"
```

En **TypeScript**, esto genera un error inmediato:

TypeScript

```
let a: string = 'hola';
a = 2; // ❌ Error: Type 'number' is not assignable to type 'string'.
```

### 3. Autodocumentación y Fiabilidad

Imagina una función para sumar. En JS, no sabes qué esperar:

JavaScript

```
function sumar(a, b) {
    return a + b;
}
```

```
// ¿Qué pasa si alguien hace sumar("1", 2)? Devuelve "12" (un string).  
// ¡No es lo que queríamos!
```

En **TS**, definimos el contrato:

TypeScript

```
function sumar(a: number, b: number): number {  
    return a + b;  
}  
// TS te impide pasarle algo que no sea un número. Es más fiable y  
robusto.
```

---

## Instalación y Configuración

### Opciones de Instalación

Dependiendo de tu flujo de trabajo, tienes varias rutas:

1. **Por Proyecto (Recomendado):** Mantiene la versión específica para tu app.

Bash

```
npm install typescript --save-dev
```

Para compilar: `npx tsc`

2. **Global:** Para probar ideas rápidas en cualquier carpeta.

Bash

```
npm install -g typescript
```

3. **Visual Studio / NuGet:** Si trabajas en ecosistemas .NET o usas Visual Studio clásico, puedes usar el gestor de paquetes NuGet:

```
Install-Package Microsoft.TypeScript.MSBuild
```

---

## Transpiladores Compatibles

A veces no usamos el compilador oficial ( `tsc` ) por velocidad, sino otras herramientas modernas:

- **Babel:** Muy popular, usa un plugin para limpiar los tipos de TS.
- **SWC:** Escrito en **Rust**, es extremadamente rápido.

- **Sucrase:** Enfocado en velocidad extrema para desarrollo.

---

## Conceptos Finales para Estudiar

- **Anotaciones de Tipos:** Es cuando le decimos explícitamente a TS qué es cada cosa (`const x: number = 5`).
- **Inferencia de Tipos:** TS es inteligente; si pones `const nombre = 'Miguel'`, él ya sabe que es un string aunque no lo digas.
- **Robustez:** TS no hace el código más corto (al revés, escribes un poco más), pero lo hace **seguro**. Menos errores en producción = menos dolores de cabeza.

---

¡Excelente continuación! Esta parte es fundamental porque entramos en la **potencia real** de TypeScript: el autocompletado, el peligro del `any` y la magia de la inferencia.

Aquí tienes la continuación de tu guía para Obsidian, organizada y visualmente clara:

---

## Herramientas y Curiosidades

TypeScript tiene una relación muy especial con sus herramientas de desarrollo:

- **VS Code:** Es la mejor herramienta para programar en TS. ¿La razón? **VS Code está escrito en TypeScript**.
- **Auto-hospedado:** TypeScript es tan potente que **TS está escrito en TypeScript**. Si vas a su repositorio de GitHub, verás que el 99% del código es `.ts`. Esto es común en lenguajes maduros (se llama *bootstrapping*).

## Instalación en diferentes Sistemas

Si necesitas instalar el compilador globalmente (`npm install -g typescript`):

| Sistema Operativo | Comando / Método                                                                      |
|-------------------|---------------------------------------------------------------------------------------|
| Fedora Linux      | <code>sudo dnf install nodejs</code> (luego el comando <code>npm</code> )             |
| Linux Mint        | <code>sudo apt install nodejs npm</code> (luego el comando <code>npm</code> )         |
| Windows 11        | Descargar instalador <code>.msi</code> de <a href="https://nodejs.org">nodejs.org</a> |

---

## La Inferencia de Tipos

TypeScript es inteligente. No siempre tienes que decirle qué tipo es cada cosa; él lo deduce por el contexto.

TypeScript

```
const a = 1          // TS infiere que 'a' es un number
const b = 2          // TS infiere que 'b' es un number
const c = a + b      // TS deduce que c es number automáticamente
```

⚠ **El límite de la inferencia Sin contexto, la inferencia no funciona. En las funciones, TS no sabe qué vas a recibir, por lo que ahí debemos ser muy específicos.**

## 🛑 El escape: Tipo `any` vs `unknown`

### 1. `any` (El forajido)

El tipo `any` le dice a TypeScript: *"Ignora todo, no revises nada"*. Es como volver a JavaScript puro.

TypeScript

```
let anyValue: any = 'hola'
anyValue.propiedadInexistente() // ✅ TS no se queja, pero romperá en el navegador.
```

- **Uso:** Se usa para "escapar" del chequeo, pero **es una mala práctica**. Existe una opción en la configuración de TS llamada `noImplicitAny` para prohibir su uso accidental.

### 2. `unknown` (El desconocido)

Es la alternativa segura a `any`. Te dice: "No sé qué es esto, así que no te dejaré hacer nada con ello hasta que asegures qué tipo es".

## 🏗️ Objetos y Autocompletado

Una de las mayores ventajas es el **IntelliSense** (autocompletado).

TypeScript

```
const persona = {
  name: 'Pepe',
```

```
    age: 30
  }
```

```
persona.age // Al poner el punto (.), VS Code te sugiere 'age' y 'name' de inmediato.
```



## Funciones y Tipado

En las funciones es donde más debemos ayudar a TypeScript para que él nos ayude a nosotros.

### Ejemplo Simple

TypeScript

```
function saludar(name: string) {
  console.log(`Hola ${name}`)
}

saludar('Pepe') // ✅ Ok
saludar(2)      // ❌ Error: No puedes pasar un número a un string
```

### Ejemplo con Objetos y Destructuración

Para objetos complejos, definimos la "forma" del objeto en los parámetros:

TypeScript

```
function saludar(persona: {name: string, age: number}) {
  const { name, age } = persona
  console.log(`Hola ${name}, tienes ${age} años`)
  return age // TS infiere que esta función retorna un 'number'
}
```



## Notas para recordar

- **TS en el código:** Cuando escribes `const n: string`, esa parte `: string` es solo para el editor y el compilador. Al pasar a JS, desaparece (o queda como comentario interno).

- **Robustez:** Si intentas acceder a una propiedad que no existe o usas un método de string en un número, TS te avisará con un subrayado rojo antes de que guardes el archivo.